AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

Course: Database System Concepts and PL/SQL

Course Code: 20CS34P

Week 8

Grouping data: Aggregate functions, GROUP BY, HAVING. Set operators: UNION, INTERSECT, MINUS

Aggregate Functions in SQL

Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary. **Grouping** is used to create subgroups of tuples before summarization.

Aggregate functions take a collection of values and return a single value as a result.

A number of built-in aggregate functions exist:

- **COUNT** function returns the number of tuples as specified in a query.
- **SUM** function returns sum of the values of tuples as specified in a query.
- **MAX** function returns the maximum value as specified in a query.
- **MIN** function returns the minimum value as specified in a query.
- **AVG** function returns the average value as specified in a query.

The functions SUM, MAX, MIN, and AVG can be applied to a set or multiset of numeric values. These functions can be used in the SELECT clause or in a HAVING clause. We illustrate the use of these functions with sample queries.

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SALARY	SUPER_SSN	DNO
JOHN	T	SMITH	12345677	01/20/1987	GPT KGF KOLAR	50000	12345677	5
franklin	T	Smile	12345679	06/12/1988	SOM KGF	30000	12345679	5
PALLAVI	C	KESHAV	12345680	06/21/0091	SOM KGF	25000	12345680	4
SANTHOSH	V	KUMAR	12345685	12/19/1986	GPTKARKALA	55000	12345685	3
SUDHA	S	RANI	12345686	02/26/1988	GPT KARKALA	65000	12345686	2
SUNEETH	S	KUMAR	12345690	04/27/1988	GPT HASSAN	15000	12345690	1

Query 1. Find the sum of the salaries of all employees, the maximum salary,the minimum salary, and the average salary.

SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)

FROM EMPLOYEE;

COUNT(*)	SUM(SALARY)	MAX(SALARY)	MIN(SALARY)	AVG(SALARY)
6	**240000**	**65000**	**15000**	**40000**

Queries 2. Retrieve the total number of employees in the company

SELECT COUNT (*) FROM EMPLOYEE;

Query 3: Retrieve the total number of employees in the 'Research' department .

SELECT COUNT (*)

FROM EMPLOYEE, DEPARTMENT

WHERE DNO=DNUMBER AND DNAME='Research';

Here the asterisk (*) refers to the rows (tuples), so COUNT (*) returns the number of rows in the result of the query.

Query 4. Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)

FROM EMPLOYEE, DEPARTMENT

WHERE DNO=DNUMBER AND DNAME='Research';

We may also use the COUNT function to count values in a column rather than tuples, as in the next example.

Query 5. Count the number of distinct salary values in the database.

SELECT COUNT (DISTINCT Salary)

FROM EMPLOYEE;

COUNT(DISTINCTSALARY)
6

If we write COUNT(SALARY) instead of COUNT(DISTINCT SALARY) in Q5, then duplicate values will not be eliminated. However, any tuples with NULL for SALARY will not be counted. In general, NULL values are **discarded** when aggregate functions are applied to a particular column (attribute).

Grouping: The GROUP BY and HAVING Clauses

In many cases we want to apply the aggregate functions to subgroups of tuples in a relation, where the subgroups are based on some attribute values. In these cases we need to **partition** the relation into nonoverlapping subsets (or **groups**) of tuples. Each group (partition) will consist of the tuples that have the same value of some attribute(s), called the **grouping attribute(s)**. We can then apply the function to each such group independently to produce summary information about each group. SQL has a **GROUP BY** clause for this purpose. The GROUP BY clause specifies the grouping attributes,which should also appear in the SELECT clause, so that the value resulting from applying each aggregate function to a group of tuples appears along with the value of the grouping attribute(s).

Query 6. For each department, retrieve the department number, the number of employees in the department, and their average salary.

SELECT Dno, COUNT (*), AVG (Salary)

FROM EMPLOYEE

GROUP BY Dno;

In Q6, the EMPLOYEE tuples are partitioned into groups—each group having the same value for the grouping attribute Dno. Hence, each group contains the employees who work in the same department. The COUNT and AVG functions are applied to each such group of tuples. SELECT clause includes only the grouping attribute and the aggregate functions to be applied on each group of tuples.

Figure 5.1(a) illustrates how grouping works on Q6; it also shows the result of Q6.

(a)

Fname	Minit	Lname	Ssn	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Borg	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

If NULLs exist in the grouping attribute, then a **separate group** is created for all tuples with a NULL value in the grouping attribute. For example, if the EMPLOYEE table had some tuples that had NULL for the grouping attribute Dno, there would be a separate group for those tuples in the result of Q6.

Query 7. For each project, retrieve the project number, the project name, and the number of employees who work on that project.

SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON

WHERE Pnumber=Pno

GROUP BY Pnumber, Pname;

HAVING clause

Sometimes we want to retrieve the values of these functions only for groups that satisfy certain conditions.SQL provides a **HAVING** clause, which can appear in conjunction with a GROUP BY clause, for this purpose. HAVING provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes. Only the groups that satisfy the condition are retrieved in the result of the query. This is illustrated by Query 8.

Query 8. For each project on which more than two employees work, retrieve the project number, the project name, and the number of employees who work on the project.

SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON

WHERE Pnumber=Pno

GROUP BY Pnumber, Pname

HAVING COUNT (*) > 2;

Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspondto the relational-algebra operations ∪, ∩, and −.

Like union, intersection, and setdifference in relational algebra, the relations participating in the operations must becompatible; that is, they must have the same set of attributes.

Let us demonstrate example queries involving the **union**,**intersect**, and **except** operations of two sets:

Query 9: The set of all customers who have anaccount at the bank, which can be derived by

SELECT CUSTOMER-NAME FROM DEPOSITOR

Query 10: The set of customers who have a loan at the bank, which can be derived by

SELECT CUSTOMER-NAME FROM BORROWER

The Union Operation

Query 11:To find all customers having a loan, an account, or both at the bank, we write

(SELECT CUSTOMER-NAME FROM DEPOSITOR)

UNION

(SELECT CUSTOMER-NAME FROM BORROWER)

The **union** operation automatically eliminates duplicates, unlike the **select** clause.If we want to retain all duplicates, we must write **union all** in place of **union**:

Q12: **(SELECT CUSTOMER-NAME FROM DEPOSITOR)**

UNION ALL

(SELECT CUSTOMER-NAME FROM BORROWER)

The number of duplicate tuples in the result is equal to the total number of duplicatesthat appear in both d and b. Thus, if Jones has three accounts and two loans at thebank, then there will be five tuples with the name Jones in the result.

The Intersect Operation

Query 13: To find all customers who have both a loan and an account at the bank, we write

(SELECT DISTINCT CUSTOMER-NAME FROM DEPOSITOR)

INTERSECT

(SELECT DISTINCT CUSTOMER-NAME FROM BORROWER)

The **intersect** operation automatically eliminates duplicates. Thus, in the query, if a customer—say, Jones—has several accounts and loans at the bank, thenJones will appear only once in the result.

If we want to retain all duplicates, we must write **intersect all** in place of **intersect**:

Q14: (SELECT CUSTOMER-NAME FROM DEPOSITOR)

INTERSECT ALL

(SELECT CUSTOMER-NAME FROM BORROWER)

The number of duplicate tuples that appear in the result is equal to the minimumnumber of duplicates in both d and b. Thus, if Jones has three accounts and two loansat the bank, then there will be two tuples with the name Jones in the result.

The Except Operation

Query 15:To find all customers who have an account but no loan at the bank, we write

(SELECT DISTINCT CUSTOMER-NAME FROM DEPOSITOR)

EXCEPT

(SELECT CUSTOMER-NAME FROM BORROWER)

The **except** operation automatically eliminates duplicates. Thus, in the precedingquery, a tuple with customer name Jones will appear (exactly once) in the result onlyif Jones has an account at the bank, but has no loan at the bank.

If we want to retain all duplicates, we must write **except all** in place of **except**:

Q16: **(` SELECT CUSTOMER-NAME FROM DEPOSITOR)**

EXCEPT ALL

(SELECT CUSTOMER-NAME FROM BORROWER)

The number of duplicate copies of a tuple in the result is equal to the number ofduplicate copies of the tuple in d minus the number of duplicate copies of the tuplein b, provided that the difference is positive. Thus, if Jones has three accounts andone loan at the bank, then there will be two tuples with the name Jones in the result.If, instead, this customer has two accounts and three loans at the bank, there will beno tuple with the name Jones in the result.

SQL also has corresponding multiset operations, which are followed by the keyword **ALL** (UNION ALL, EXCEPT ALL, INTERSECT ALL). Their results are multisets (duplicates are not eliminated). Basically, each tuple—whether it is a duplicate or not—is considered as a different tuple when applying these operations.

(a) R

A
a1
a2
a2
a3

S

A
a1
a2
a4
a5

(b) T

A
a1
a1
a2
a2
a2
a3
a4
a5

(c) T

A
a2
a3

(d) T

A
a1
a2

Figure 4.5
The results of SQL multiset operations. (a) Two tables, R(A) and S(A). (b) R(A) UNION ALL S(A). (c) R(A) EXCEPT ALL S(A). (d) R(A) INTERSECT ALL S(A).